I am creating APIs for a Mobility Solution company. This company offer many services including vehicle valuation. I have come up with a one core system which bring together all those service, where I am using micro-services to represent each service. The programming technologies used include :-

.Backend: Go 1.21, Gin HTTP framework

- **Database**: PostgreSQL with GORM
- **Cache**: Redis
- **Message Queue**: Apache Kafka
- **Authentication**: JWT tokens
- **Storage**: Firebase Storage
- **Notifications**: Firebase Cloud Messaging
- **Documentation**: Swagger/OpenAPI
- **Containerization**: Docker

The project micro-service structure is as follow. The below is user-management micro-service.

```
internal/
├── vehicle-service/
│   ├── handlers/       # HTTP handlers
│   ├── models/         # Data models
│   ├── repositories/   # Data access layer
│   ├── services/       # Business logic
│   └── routes/         # Route definitions
├── shared/
│   ├── database/       # Database connection
│   ├── redis/          # Redis client
│   ├── kafka/          # Kafka producer/consumer
│   ├── firebase/       # Firebase services
│   └── utils/          # Utility functions
└── pkg/
    ├── auth/           # JWT authentication
    ├── response/       # API response helpers
    └── validation/     # Input validation
```

The main.go file for every microservice is outside the internal package.

Here are sample codes of how I have implemented a model for vehicle-service. This model is able to auto generate database tables. Migrations have been implemented in main.go

1. vehicle_model.go

```
package models

import (
"time"

"github.com/google/uuid"
"gorm.io/gorm"
)

type VehicleType string
type FuelType string
type TransmissionType string
type VehicleStatus string

const (
VehicleTypeCar VehicleType = "car"
VehicleTypeMotorcycle VehicleType = "motorcycle"
VehicleTypeTruck VehicleType = "truck"
VehicleTypeBus VehicleType = "bus"

FuelTypePetrol FuelType = "petrol"
FuelTypeDiesel FuelType = "diesel"
FuelTypeElectric FuelType = "electric"
FuelTypeHybrid FuelType = "hybrid"

TransmissionManual TransmissionType = "manual"
TransmissionAutomatic TransmissionType = "automatic"
TransmissionCVT TransmissionType = "cvt"

VehicleStatusActive VehicleStatus = "active"
VehicleStatusInactive VehicleStatus = "inactive"
VehicleStatusSold VehicleStatus = "sold"
)

// Customer represents minimal customer data synced from user service
type Customer struct {
ID string `json:"id" gorm:"type:uuid;primary_key"`
Email string `json:"email" gorm:"not null"`
Phone string `json:"phone" gorm:"not null"`
FirstName string `json:"first_name" gorm:"not null"`
```

```
LastName string `json:"last_name" gorm:"not null"`
Status string `json:"status" gorm:"not null"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
}

type VehicleMake struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"unique;not null"`
Country string `json:"country"`
LogoURL *string `json:"logo_url,omitempty"`
IsActive bool `json:"is_active" gorm:"default:true"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

Models []VehicleModel `json:"models" gorm:"foreignKey:MakeID"`
}

type VehicleModel struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
MakeID string `json:"make_id" gorm:"not null"`
Name string `json:"name" gorm:"not null"`
Description *string `json:"description,omitempty"` // ADDED: Description field
VehicleType VehicleType `json:"vehicle_type" gorm:"not null"`
StartYear int `json:"start_year"`
EndYear *int `json:"end_year,omitempty"`
IsActive bool `json:"is_active" gorm:"default:true"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

Make VehicleMake `json:"make" gorm:"foreignKey:MakeID"`
CustomerVehicles []CustomerVehicle `json:"customer_vehicles"
gorm:"foreignKey:ModelID"`
}

type CustomerVehicle struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
CustomerID string `json:"customer_id" gorm:"not null"`
MakeID string `json:"make_id" gorm:"not null"`
ModelID string `json:"model_id" gorm:"not null"`
RegistrationNo string `json:"registration_no" gorm:"unique;not null"`
YearOfManufacture int `json:"year_of_manufacture" gorm:"not null"`
Color string `json:"color"`
ChassisNumber *string `json:"chassis_number,omitempty"`
```

```
EngineNumber *string `json:"engine_number,omitempty"`
FuelType FuelType `json:"fuel_type" gorm:"not null"` // ADDED: FuelType field
Transmission TransmissionType `json:"transmission" gorm:"not null"` // ADDED:
Transmission field
EngineCapacity *float64 `json:"engine_capacity,omitempty"`
Mileage *int `json:"mileage,omitempty"`
PurchaseDate *time.Time `json:"purchase_date,omitempty"`
Status VehicleStatus `json:"status" gorm:"default:active"`
Notes *string `json:"notes,omitempty"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

Customer Customer `json:"customer" gorm:"foreignKey:CustomerID"`
Make VehicleMake `json:"make" gorm:"foreignKey:MakeID"`
Model VehicleModel `json:"model" gorm:"foreignKey:ModelID"`
}

type VehicleImage struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
VehicleID string `json:"vehicle_id" gorm:"not null"`
ImageURL string `json:"image_url" gorm:"not null"`
ImageType string `json:"image_type"` // front, back, side, interior, engine, documents
IsPrimary bool `json:"is_primary" gorm:"default:false"`
CreatedAt time.Time `json:"created_at"`

Vehicle CustomerVehicle `json:"-" gorm:"foreignKey:VehicleID"`
}

func (c *Customer) BeforeCreate(tx *gorm.DB) error {
if c.ID == "" {
c.ID = uuid.New().String()
}
return nil
}

func (vm *VehicleMake) BeforeCreate(tx *gorm.DB) error {
if vm.ID == "" {
vm.ID = uuid.New().String()
}
return nil
}

func (vmodel *VehicleModel) BeforeCreate(tx *gorm.DB) error {
if vmodel.ID == "" {
```

```
vmodel.ID = uuid.New().String()
}
return nil
}

func (cv *CustomerVehicle) BeforeCreate(tx *gorm.DB) error {
if cv.ID == "" {
cv.ID = uuid.New().String()
}
return nil
}
```

2. routes.go

```
package routes

import (
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/middleware"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/handlers"
"github.com/gin-gonic/gin"
swaggerFiles "github.com/swaggo/files"
ginSwagger "github.com/swaggo/gin-swagger"
)

func SetupRoutes(
r *gin.Engine,
vehicleHandler *handlers.VehicleHandler,
authMiddleware *middleware.AuthMiddleware,
) {
// Add CORS middleware
r.Use(middleware.CORSMiddleware())

// Add logging middleware
r.Use(middleware.LoggingMiddleware())

// Health check
r.GET("/health", func(c *gin.Context) {
c.JSON(200, gin.H{
"status": "healthy",
"service": "vehicle-service",
```

```
"timestamp": time.Now(),
})
})

// Swagger documentation
r.GET("/docs/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))

// API routes
api := r.Group("/api/v1")

// Public vehicle reference data (no auth required)
vehicles := api.Group("/vehicles")
{
vehicles.GET("/makes", vehicleHandler.GetAllMakes)
//vehicles.GET("/makes/:id", vehicleHandler.GetMakeByID)
//vehicles.GET("/models", vehicleHandler.GetAllModels)
vehicles.GET("/makes/:make_id/models", vehicleHandler.GetModelsByMake)
vehicles.GET("/registration/:registration", vehicleHandler.GetVehicleByRegistration)
}

// Protected routes
protected := api.Group("/")
protected.Use(authMiddleware.RequireAuth())
{
// Customer vehicle operations
protected.POST("/vehicles/customer", vehicleHandler.CreateCustomerVehicle)
//protected.GET("/vehicles/:id", vehicleHandler.GetVehicleByID)

// Customer-specific routes
customers := protected.Group("/customers")
{
customers.GET("/:customer_id/vehicles", vehicleHandler.GetCustomerVehicles)
}

// Admin only routes
admin := protected.Group("/admin")
admin.Use(authMiddleware.RequireRole("admin", "super_admin"))
{
admin.POST("/vehicles/makes", vehicleHandler.CreateMake)
admin.POST("/vehicles/models", vehicleHandler.CreateModel)
}
}
}
```

3. main.go

```
package main

import (
"context"
"log"
"net/http"
"os"
"os/signal"
"syscall"
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/database"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/kafka"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/redis"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/config"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/middleware"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/handlers"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/models"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/repositories"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/routes"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/services"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/auth"
"github.com/gin-gonic/gin"
"github.com/joho/godotenv"
"gorm.io/gorm"
)

// @title AA Core System - Vehicle Service API
// @version 1.0
// @description Vehicle management service for AA Core System
// @contact.name API Support
// @contact.email support@aa.co.ke
// @host localhost:8082
```

```
// @BasePath /api/v1
// @securityDefinitions.apikey BearerAuth
// @in header
// @name Authorization
func main() {
// Load environment variables
if err := godotenv.Load(); err != nil {
log.Println("No .env file found, using system environment variables")
}

// Load configuration
cfg, err := config.LoadConfig("./configs")
if err != nil {
log.Fatalf("Failed to load config: %v", err)
}

// Set Gin mode
gin.SetMode(cfg.Server.Mode)

// Initialize database
dbConfig := database.Config{
Host: cfg.Database.Host,
Port: cfg.Database.Port,
User: cfg.Database.User,
Password: cfg.Database.Password,
DBName: "aa_vehicle_service", // Separate database
SSLMode: cfg.Database.SSLMode,
MaxOpenConns: cfg.Database.MaxOpenConns,
MaxIdleConns: cfg.Database.MaxIdleConns,
ConnMaxLifetime: cfg.Database.ConnMaxLifetime,
}

db, err := database.NewPostgresConnection(dbConfig)
if err != nil {
log.Fatalf("Failed to connect to database: %v", err)
}

// Run migrations
if err := runMigrations(db); err != nil {
log.Fatalf("Failed to run migrations: %v", err)
}

// Initialize Redis
redisConfig := redis.Config{
```

```
Host: cfg.Redis.Host,
Port: cfg.Redis.Port,
Password: cfg.Redis.Password,
DB: 2, // Different Redis DB for vehicle service
PoolSize: cfg.Redis.PoolSize,
}

redisClient, err := redis.NewRedisClient(redisConfig)
if err != nil {
log.Fatalf("Failed to connect to Redis: %v", err)
}

// Initialize Kafka
kafkaConfig := kafka.Config{
Brokers: cfg.Kafka.Brokers,
Topics: map[string]string{
"user_events": "user-events",
"vehicle_events": "vehicle-events",
},
}

kafkaProducer := kafka.NewProducer(kafkaConfig)
defer kafkaProducer.Close()

// Initialize JWT manager
jwtManager := auth.NewJWTManager(
cfg.JWT.Secret,
cfg.JWT.AccessTokenDuration,
cfg.JWT.RefreshTokenDuration,
)

// Initialize repositories
vehicleRepo := repositories.NewVehicleRepository(db)

// Initialize services
vehicleService := services.NewVehicleService(vehicleRepo, kafkaProducer)

// Initialize middleware
authMiddleware := middleware.NewAuthMiddleware(jwtManager, nil, redisClient)

// Initialize handlers
vehicleHandler := handlers.NewVehicleHandler(vehicleService)

// Initialize Gin router
```

```
router := gin.New()
router.Use(gin.Recovery())

// Setup routes
routes.SetupRoutes(router, vehicleHandler, authMiddleware)

// Start Kafka consumers in background
go startKafkaConsumers(kafkaConfig, vehicleService)

// Seed initial data
go func() {
time.Sleep(5 * time.Second) // Wait for service to be ready
if err := seedInitialData(vehicleService); err != nil {
log.Printf("Failed to seed initial data: %v", err)
}
}()

// Create HTTP server
srv := &http.Server{
Addr: ":8082", // Different port for vehicle service
Handler: router,
ReadTimeout: cfg.Server.ReadTimeout,
WriteTimeout: cfg.Server.WriteTimeout,
}

// Start server in a goroutine
go func() {
log.Printf("Starting vehicle service on port 8082")
if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
log.Fatalf("Failed to start server: %v", err)
}
}()

// Wait for interrupt signal to gracefully shutdown the server
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit

log.Println("Shutting down vehicle service...")

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
```

```
log.Fatalf("Server forced to shutdown: %v", err)
}

log.Println("Vehicle service shutdown complete")
}

func runMigrations(db *gorm.DB) error {
log.Println("Running vehicle service migrations...")

err := db.AutoMigrate(
&models.Customer{},
&models.VehicleMake{},
&models.VehicleModel{},
&models.CustomerVehicle{},
)

if err != nil {
return err
}

log.Println("Vehicle service migrations completed successfully")
return nil
}

func startKafkaConsumers(kafkaConfig kafka.Config, vehicleService
*services.VehicleService) {
consumer := kafka.NewConsumer(kafkaConfig, "vehicle-service")

// Handle user events from user-management service
go func() {
err := consumer.ConsumeEvents("user_events", func(event kafka.Event) error {
switch event.Type {
case "user.registered":
return vehicleService.HandleCustomerCreated(event)
case "user.email_verified":
return vehicleService.HandleCustomerVerified(event)
default:
log.Printf("Unhandled event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume user events: %v", err)
}
```

```
}()
}

func seedInitialData(vehicleService *services.VehicleService) error {
log.Println("Seeding initial vehicle data...")

// Popular vehicle makes
makes := []services.CreateMakeRequest{
{Name: "Toyota", Country: "Japanese"},
{Name: "Honda", Country: "Japanese"},
{Name: "Nissan", Country: "Japanese"},
{Name: "Mazda", Country: "Japanese"},
{Name: "Subaru", Country: "Japanese"},
{Name: "Mitsubishi", Country: "Japanese"},
{Name: "Suzuki", Country: "Japanese"},
{Name: "Mercedes-Benz", Country: "German"},
{Name: "BMW", Country: "German"},
{Name: "Audi", Country: "German"},
{Name: "Volkswagen", Country: "German"},
{Name: "Ford", Country: "American"},
{Name: "Chevrolet", Country: "American"},
{Name: "Hyundai", Country: "Korean"},
{Name: "Kia", Country: "Korean"},
{Name: "Peugeot", Country: "French"},
{Name: "Renault", Country: "French"},
{Name: "Volvo", Country: "Swedish"},
{Name: "Land Rover", Country: "British"},
{Name: "Isuzu", Country: "Japanese"},
}

// Create makes
for _, makeReq := range makes {
if _, err := vehicleService.CreateMake(makeReq); err != nil {
continue
}
}

log.Println("Initial vehicle data seeded successfully")
return nil
}
```